

TYPES AND DECLARATIONS

1. What is the difference between a declaration and a definition?

TYPES AND DECLARATIONS

2. What are fundamental types? Give examples of integer, floating-point, character, Boolean, and `void` types.

TYPES AND DECLARATIONS

3. Why are fundamental type sizes implementation-defined, and how do `<stdint>` types help portability?

TYPES AND DECLARATIONS

4. What is a scope? Explain local, class, namespace, global, and statement scope.

TYPES AND DECLARATIONS

5. What are hiding and shadowing? Give an example.

TYPES AND DECLARATIONS

6. What is the `auto` type specifier, and when should it be used?

TYPES AND DECLARATIONS

7. What is an lvalue? What is an rvalue?

TYPES AND DECLARATIONS

8. What happens if a name is declared without initialization?

TYPES AND DECLARATIONS

9. What is a C++ object?

TYPES AND DECLARATIONS

10. What is object lifetime?

TYPES AND DECLARATIONS

11. Compare automatic, static, free-store, `thread_local`, and temporary object lifetimes.

TYPES AND DECLARATIONS

12. What is brace initialization, and why does it help avoid narrowing?

TYPES AND DECLARATIONS

13. What is the difference between `const` and `constexpr`?

TYPES AND DECLARATIONS

14. What is a type alias, and why can aliases improve readability?

TYPES AND DECLARATIONS

15. [Prof] After the operations below, which is the value of `c`?

```
uint_fast8_t a = 21;  
uint_fast8_t b = 11;  
uint_fast8_t c = a ^ b;
```

POINTERS, ARRAYS AND REFERENCES

1. What is a pointer?

POINTERS, ARRAYS AND REFERENCES

2. What operations are possible with pointers?

POINTERS, ARRAYS AND REFERENCES

3. What is a raw pointer, and why does it not necessarily express ownership?

POINTERS, ARRAYS AND REFERENCES

4. What is `void*`, and why does it lose type information?

POINTERS, ARRAYS AND REFERENCES

5. What is `nullptr`, and why is it better than 0 or `NULL`?

POINTERS, ARRAYS AND REFERENCES

6. What is the behavior of `const` with pointers?

POINTERS, ARRAYS AND REFERENCES

7. **[Prof]** What is the difference between `const std::string* s` and `std::string* const s`?

POINTERS, ARRAYS AND REFERENCES

8. What is an array, and how can it be initialized?

POINTERS, ARRAYS AND REFERENCES

9. What does it mean that arrays are contiguous?

POINTERS, ARRAYS AND REFERENCES

10. What happens when an array is passed to a function?

POINTERS, ARRAYS AND REFERENCES

11. What is pointer arithmetic, and when is it valid?

POINTERS, ARRAYS AND REFERENCES

12. What is an lvalue reference?

POINTERS, ARRAYS AND REFERENCES

13. What is an rvalue reference?

POINTERS, ARRAYS AND REFERENCES

14. [**Prof**] What is a reference? Provide an example.

POINTERS, ARRAYS AND REFERENCES

15. How do references differ from pointers?

POINTERS, ARRAYS AND REFERENCES

16. Why are rvalue references important for move semantics?

STRUCTURES AND ENUMERATORS

1. What are primitive user-defined types?

STRUCTURES AND ENUMERATORS

2. What is a `struct`?

STRUCTURES AND ENUMERATORS

3. How does a `struct` differ from a `class` by default access?

STRUCTURES AND ENUMERATORS

4. What is aggregate initialization?

STRUCTURES AND ENUMERATORS

5. What is an invariant, and can a `struct` enforce one?

STRUCTURES AND ENUMERATORS

6. What is Plain Old Data (POD)?

STRUCTURES AND ENUMERATORS

7. What is memory padding, and why can member order affect object size?

STRUCTURES AND ENUMERATORS

8. What is a bitfield, and when can it be useful?

STRUCTURES AND ENUMERATORS

9. What is an `enum class`?

STRUCTURES AND ENUMERATORS

10. Why is `enum class` safer than plain `enum`?

STRUCTURES AND ENUMERATORS

11. What does it mean that `enum class` is scoped and strongly typed?

STRUCTURES AND ENUMERATORS

12. How can explicit enum values be useful?

STATEMENTS

1. What is a statement in C++?

STATEMENTS

2. Why is a declaration also a statement?

STATEMENTS

3. Why should variables be declared near first use and initialized immediately?

STATEMENTS

4. How does an `if` statement work?

STATEMENTS

5. How does the ternary conditional operator work, and when is it appropriate?

STATEMENTS

6. **[Prof]** Explain how the `switch-case` statement works, providing an example.

STATEMENTS

7. What is the role of `break` in a `switch` statement?

STATEMENTS

8. How does a range-for loop work?

STATEMENTS

9. Compare `for`, `while`, and `do while` loops.

STATEMENTS

10. What are `break`, `continue`, and `return`, and how do they affect control flow?

STATEMENTS

11. What is short-circuit evaluation for `&&` and `||`?

STATEMENTS

12. What makes a loop condition dangerous or unclear?

NAMESPACES AND CODE MODULARIZATION

1. What problem do namespaces solve?

NAMESPACES AND CODE MODULARIZATION

2. How do namespaces support modularity?

NAMESPACES AND CODE MODULARIZATION

3. What is name qualification with ::?

NAMESPACES AND CODE MODULARIZATION

4. What is the difference between a `using` declaration and a `using namespace` directive?

NAMESPACES AND CODE MODULARIZATION

5. Why can `using namespace std;` be dangerous in headers?

NAMESPACES AND CODE MODULARIZATION

6. What is an interface in modular code?

NAMESPACES AND CODE MODULARIZATION

7. Why should implementation details be hidden?

NAMESPACES AND CODE MODULARIZATION

8. How do namespaces, classes, and functions work together to organize code?

NAMESPACES AND CODE MODULARIZATION

9. What is argument-dependent lookup?

NAMESPACES AND CODE MODULARIZATION

10. How can code be split across headers and source files?

FUNCTIONS

1. What is a function declaration?

FUNCTIONS

2. What is a function definition?

FUNCTIONS

3. What are function name, argument list, and return type?

FUNCTIONS

4. What does `void` mean as function return type?

FUNCTIONS

5. What is the difference between passing by value, by pointer, and by reference?

FUNCTIONS

6. [Prof] We want to pass a string `s`, an integer `i`, and a double `d` as input parameters to a function. They cannot be modified by the function, and the function should be efficient. How do we pass them?

```
void fun(... s, ... i, ... d);
```

FUNCTIONS

7. When should small values be passed by value?

FUNCTIONS

8. When should large read-only objects be passed by `const` reference?

FUNCTIONS

9. When should a function use pointer parameters?

FUNCTIONS

10. What is function overloading?

FUNCTIONS

11. Why can functions not be overloaded only by return type?

FUNCTIONS

12. What is overload resolution?

FUNCTIONS

13. What are preconditions and postconditions?

FUNCTIONS

14. What is a function pointer?

FUNCTIONS

15. What are macros, and why are they less type-safe than functions or constants?

FUNCTIONS

16. What do `inline`, `constexpr`, and `noexcept` mean for functions?

CLASSES

1. What is a class?

CLASSES

2. What are representation, behavior, invariant, and interface?

CLASSES

3. What is the difference between public and private members?

CLASSES

4. Why does access control protect invariants?

CLASSES

5. What is a member function?

CLASSES

6. How can member functions be defined inside or outside class declaration?

CLASSES

7. What is a constructor?

CLASSES

8. What does `explicit` prevent?

CLASSES

9. What is a const member function?

CLASSES

10. What is logical constness?

CLASSES

11. What is `mutable`, and when can it be justified?

CLASSES

12. What is `this`?

CLASSES

13. What are static data members and static member functions?

CLASSES

14. What is a concrete class?

CLASSES

15. How should a class expose a clear interface without exposing representation?

COMPILER, PREPROCESSOR AND LINKER

1. **[Prof]** Explain the role of the preprocessor, compiler, and linker.

COMPILER, PREPROCESSOR AND LINKER

2. What is a translation unit?

COMPILER, PREPROCESSOR AND LINKER

3. What does the preprocessor do with `#include`?

COMPILER, PREPROCESSOR AND LINKER

4. What are include guards, and why are they necessary?

COMPILER, PREPROCESSOR AND LINKER

5. What is conditional compilation?

COMPILER, PREPROCESSOR AND LINKER

6. What is the difference between compiler errors and linker errors?

COMPILER, PREPROCESSOR AND LINKER

7. What does compiler optimization do?

COMPILER, PREPROCESSOR AND LINKER

8. What are object files?

COMPILER, PREPROCESSOR AND LINKER

9. What symbols does the linker resolve?

COMPILER, PREPROCESSOR AND LINKER

10. What is the difference between static and dynamic libraries?

COMPILER, PREPROCESSOR AND LINKER

11. Why are headers not compiled independently?

COMPILER, PREPROCESSOR AND LINKER

12. Why do template definitions often need to be visible in headers?

CONSTRUCTION, CLEANUP, COPY AND MOVE

1. What is object life cycle in C++?

CONSTRUCTION, CLEANUP, COPY AND MOVE

2. What does a constructor establish?

CONSTRUCTION, CLEANUP, COPY AND MOVE

3. What does a destructor do?

CONSTRUCTION, CLEANUP, COPY AND MOVE

4. What is RAII, and why is it important?

CONSTRUCTION, CLEANUP, COPY AND MOVE

5. What is the construction and destruction order for class members?

CONSTRUCTION, CLEANUP, COPY AND MOVE

6. Compare default initialization, value initialization, direct initialization, copy initialization, and list initialization.

CONSTRUCTION, CLEANUP, COPY AND MOVE

7. Why is brace initialization often preferred?

CONSTRUCTION, CLEANUP, COPY AND MOVE

8. When can initializer-list constructors affect overload selection?

CONSTRUCTION, CLEANUP, COPY AND MOVE

9. What is aggregate initialization?

CONSTRUCTION, CLEANUP, COPY AND MOVE

10. What is copy construction?

CONSTRUCTION, CLEANUP, COPY AND MOVE

11. What is copy assignment?

CONSTRUCTION, CLEANUP, COPY AND MOVE

12. What should copying preserve: equivalence, independence, or both?

CONSTRUCTION, CLEANUP, COPY AND MOVE

13. Why is default shallow copy dangerous for owning pointers?

CONSTRUCTION, CLEANUP, COPY AND MOVE

14. What is move construction?

CONSTRUCTION, CLEANUP, COPY AND MOVE

15. What is move assignment?

CONSTRUCTION, CLEANUP, COPY AND MOVE

16. What state must a moved-from object be left in?

CONSTRUCTION, CLEANUP, COPY AND MOVE

17. Which default operations can the compiler generate?

CONSTRUCTION, CLEANUP, COPY AND MOVE

18. What does `= delete` mean?

CONSTRUCTION, CLEANUP, COPY AND MOVE

19. What is the Rule of Three/Five idea?

CONSTRUCTION, CLEANUP, COPY AND MOVE

20. Why do resource-owning classes need careful copy and move semantics?

MEMORY MANAGEMENT

1. What are the main memory regions in a C++ program?

MEMORY MANAGEMENT

2. What is stack memory?

MEMORY MANAGEMENT

3. What is free-store or heap memory?

MEMORY MANAGEMENT

4. What are static and global storage areas?

MEMORY MANAGEMENT

5. How do `new` and `delete` work?

MEMORY MANAGEMENT

6. What is the difference between `new`, `delete`, `new[]`, and `delete[]`?

MEMORY MANAGEMENT

7. What is a memory leak?

MEMORY MANAGEMENT

8. What is a dangling pointer?

MEMORY MANAGEMENT

9. What is double deletion?

MEMORY MANAGEMENT

10. Why is manual memory management error-prone?

MEMORY MANAGEMENT

11. What is resource management?

MEMORY MANAGEMENT

12. How does RAII make cleanup automatic?

MEMORY MANAGEMENT

13. What is a handle class?

MEMORY MANAGEMENT

14. Why do handle classes need correct copy and move behavior?

MEMORY MANAGEMENT

15. Why are smart pointers preferred over raw owning pointers?

DERIVED CLASSES AND CLASS HIERARCHIES

1. What is inheritance?

DERIVED CLASSES AND CLASS HIERARCHIES

2. What is an is-a relationship?

DERIVED CLASSES AND CLASS HIERARCHIES

3. What is the difference between implementation inheritance and interface inheritance?

DERIVED CLASSES AND CLASS HIERARCHIES

4. What does public inheritance mean for substitutability?

DERIVED CLASSES AND CLASS HIERARCHIES

5. What is object slicing?

DERIVED CLASSES AND CLASS HIERARCHIES

6. How are base and derived constructors/destructors called?

DERIVED CLASSES AND CLASS HIERARCHIES

7. What is a virtual function?

DERIVED CLASSES AND CLASS HIERARCHIES

8. What is runtime polymorphism through pointers and references?

DERIVED CLASSES AND CLASS HIERARCHIES

9. What does `override` do?

DERIVED CLASSES AND CLASS HIERARCHIES

10. What does `final` do?

DERIVED CLASSES AND CLASS HIERARCHIES

11. What is a pure virtual function?

DERIVED CLASSES AND CLASS HIERARCHIES

12. What is an abstract class?

DERIVED CLASSES AND CLASS HIERARCHIES

13. What is the difference between `public`, `protected`, and `private` inheritance or access?

DERIVED CLASSES AND CLASS HIERARCHIES

14. What problem can multiple inheritance create?

DERIVED CLASSES AND CLASS HIERARCHIES

15. What is a virtual base class?

OPERATOR OVERLOADING

1. What is operator overloading?

OPERATOR OVERLOADING

2. Why should overloaded operators preserve intuitive meaning?

OPERATOR OVERLOADING

3. Which operators cannot be overloaded?

OPERATOR OVERLOADING

4. What is the difference between member and non-member operator overloads?

OPERATOR OVERLOADING

5. When must an operator be a non-member?

OPERATOR OVERLOADING

6. How are binary operators represented as member and non-member functions?

OPERATOR OVERLOADING

7. How are unary operators represented as member and non-member functions?

OPERATOR OVERLOADING

8. Why should built-in types usually be passed by value and user-defined types by `const` reference?

OPERATOR OVERLOADING

9. How is `operator<<` usually implemented?

OPERATOR OVERLOADING

10. Why should `operator<<` return `std::ostream&`?

OPERATOR OVERLOADING

11. When is `friend` useful for operator overloading?

OPERATOR OVERLOADING

12. What is `operator[]`, and what should it return?

OPERATOR OVERLOADING

13. What is `operator()`, and how does it create a functor?

OPERATOR OVERLOADING

14. Why are functors useful with standard algorithms?

RUNTIME POLYMORPHISM AND CASTS

1. What is runtime polymorphism?

RUNTIME POLYMORPHISM AND CASTS

2. What is an upcast?

RUNTIME POLYMORPHISM AND CASTS

3. What is a downcast?

RUNTIME POLYMORPHISM AND CASTS

4. What is a crosscast?

RUNTIME POLYMORPHISM AND CASTS

5. What is RTTI?

RUNTIME POLYMORPHISM AND CASTS

6. What does `dynamic_cast` do?

RUNTIME POLYMORPHISM AND CASTS

7. What happens when `dynamic_cast` fails for a pointer?

RUNTIME POLYMORPHISM AND CASTS

8. What happens when `dynamic_cast` fails for a reference?

RUNTIME POLYMORPHISM AND CASTS

9. Why should RTTI be used sparingly?

RUNTIME POLYMORPHISM AND CASTS

10. Why is a virtual function often better than checking actual derived type?

RUNTIME POLYMORPHISM AND CASTS

11. What is `static_cast` used for?

RUNTIME POLYMORPHISM AND CASTS

12. What is `const_cast` used for?

RUNTIME POLYMORPHISM AND CASTS

13. What is `reinterpret_cast` used for?

RUNTIME POLYMORPHISM AND CASTS

14. What are smart pointer casts, such as `dynamic_pointer_cast`?

TEMPLATES

1. What is generic programming?

TEMPLATES

2. What is a template?

TEMPLATES

3. What is a class template?

TEMPLATES

4. What is a function template?

TEMPLATES

5. What is template instantiation?

TEMPLATES

6. Why do templates usually have no runtime overhead?

TEMPLATES

7. Why do template definitions usually live in headers?

TEMPLATES

8. What can a class template contain?

TEMPLATES

9. What is a member template?

TEMPLATES

10. What is a template specialization?

TEMPLATES

11. What is a type alias for a template type?

TEMPLATES

12. What is a variadic template?

TEMPLATES

13. What problem do concepts solve in C++20?

TEMPLATES

14. Why are template error messages often difficult?

TEMPLATES

15. How are templates used in the Standard Library?

SMART POINTERS

1. What problem do raw pointers create for ownership and lifetime?

SMART POINTERS

2. What is RAII, and how do smart pointers implement it?

SMART POINTERS

3. What is `std::unique_ptr`?

SMART POINTERS

4. What is exclusive ownership?

SMART POINTERS

5. When should `unique_ptr` be used?

SMART POINTERS

6. **[Prof]** When should I use `unique_ptr`, `shared_ptr`, and `weak_ptr`?

SMART POINTERS

7. **[Prof]** How do I pass a `unique_ptr` to functions?

SMART POINTERS

8. What is `std::move`, and why is it needed with `unique_ptr`?

SMART POINTERS

9. What do `release()` and `reset()` do on a `unique_ptr`?

SMART POINTERS

10. What is `std::shared_ptr`?

SMART POINTERS

11. What is reference counting?

SMART POINTERS

12. What does `use_count()` mean?

SMART POINTERS

13. When is `shared_ptr` appropriate, and when is it overused?

SMART POINTERS

14. What is `std::weak_ptr`?

SMART POINTERS

15. **[Prof]** How do I use a `weak_ptr`?

SMART POINTERS

16. How does `weak_ptr` help avoid ownership cycles?

SMART POINTERS

17. What does `lock()` do on a `weak_ptr`?

SMART POINTERS

18. What are common wrong uses of smart pointers?

SMART POINTERS

19. [Prof] What is an lvalue?

SMART POINTERS

20. [Prof] What is an rvalue?

STANDARD LIBRARY

1. What is the C++ Standard Library?

STANDARD LIBRARY

2. What categories of facilities does the Standard Library provide?

STANDARD LIBRARY

3. What is the difference between containers, iterators, and algorithms?

STANDARD LIBRARY

4. Why is `std::vector` usually the default sequence container?

STANDARD LIBRARY

5. Compare `vector`, `deque`, `list`, and `forward_list`.

STANDARD LIBRARY

6. Compare ordered associative containers and unordered associative containers.

STANDARD LIBRARY

7. What are `map`, `set`, `unordered_map`, and `unordered_set`?

STANDARD LIBRARY

8. What are container adaptors such as `stack`, `queue`, and `priority_queue`?

STANDARD LIBRARY

9. What is `std::array`?

STANDARD LIBRARY

10. What is an iterator?

STANDARD LIBRARY

11. What do `begin()` and `end()` represent?

STANDARD LIBRARY

12. Compare input, output, forward, bidirectional, and random-access iterators.

STANDARD LIBRARY

13. What is an algorithm in the Standard Library?

STANDARD LIBRARY

14. Why do algorithms operate on iterator ranges rather than directly on containers?

STANDARD LIBRARY

15. Give examples of non-modifying and modifying algorithms.

STANDARD LIBRARY

16. What is `std::string`, and why is it container-like?

STANDARD LIBRARY

17. What are C++ streams?

STANDARD LIBRARY

18. Compare `cout`, `cerr`, `clog`, and `cin`.

STANDARD LIBRARY

19. Compare `ifstream`, `ofstream`, and `fstream`.

STANDARD LIBRARY

20. What are string streams, and when are they useful?

SOCKET PROGRAMMING

1. What is a socket?

SOCKET PROGRAMMING

2. Why is a socket represented as a file descriptor in POSIX?

SOCKET PROGRAMMING

3. What is the difference between an IP address and a port?

SOCKET PROGRAMMING

4. What is the difference between UDP and TCP?

SOCKET PROGRAMMING

5. What is `SOCK_DGRAM`?

SOCKET PROGRAMMING

6. What is `SOCK_STREAM`?

SOCKET PROGRAMMING

7. What headers are needed for basic POSIX socket programming?

SOCKET PROGRAMMING

8. What is `sockaddr_in`, and which fields must be filled?

SOCKET PROGRAMMING

9. Why are `htons()` and `htonl()` needed?

SOCKET PROGRAMMING

10. What does `inet_pton()` do?

SOCKET PROGRAMMING

11. What does `socket()` return?

SOCKET PROGRAMMING

12. What does `bind()` do?

SOCKET PROGRAMMING

13. What does `listen()` do?

SOCKET PROGRAMMING

14. What does `accept()` do?

SOCKET PROGRAMMING

15. What does `connect()` do?

SOCKET PROGRAMMING

16. Compare `send()/recv()` with `sendto()/recvfrom()`.

SOCKET PROGRAMMING

17. What is typical UDP server/client flow?

SOCKET PROGRAMMING

18. What is typical TCP server/client flow?

SOCKET PROGRAMMING

19. Why must return values from socket calls be checked?

SOCKET PROGRAMMING

20. What is SIGPIPE, and why can it happen when sending on a closed socket?

SOCKET PROGRAMMING

21. What does SO_REUSEADDR do?

SOCKET PROGRAMMING

22. Why is `close()` important for sockets?

SOCKET PROGRAMMING

23. How can threads be useful in a socket chat program?

THREADS AND LAMBDA

1. What is parallel programming?

THREADS AND LAMBDA

2. What is the difference between a process and a thread?

THREADS AND LAMBDA

3. Why are threads lighter than processes?

THREADS AND LAMBDA

4. What memory do threads share?

THREADS AND LAMBDA

5. [Prof] What's going on when multiple threads are running in parallel?

THREADS AND LAMBDA

6. How does `std::thread` start a task?

THREADS AND LAMBDA

7. Why must a `std::thread` be joined or detached before destruction?

THREADS AND LAMBDA

8. What is the difference between `join()` and `detach()`?

THREADS AND LAMBDA

9. Why is `join()` usually preferred?

THREADS AND LAMBDA

10. How are thread arguments passed by default?

THREADS AND LAMBDA

11. When should `std::ref` be used?

THREADS AND LAMBDA

12. How can ownership be transferred to a thread?

THREADS AND LAMBDA

13. How can `unique_ptr` or `shared_ptr` be used with threads?

THREADS AND LAMBDA

14. What is a lambda function?

THREADS AND LAMBDA

15. What is a lambda capture list?

THREADS AND LAMBDA

16. Compare capture by value and capture by reference.

THREADS AND LAMBDA

17. Why is lifetime important with lambda captures in threads?

THREADS AND LAMBDA

18. What is a callable object?

THREADS AND LAMBDA

19. Why can lambdas be useful with standard algorithms?

THREADS AND LAMBDA

20. Why does sharing memory between threads create risk?

INTER-THREAD COMMUNICATION

1. What is shared memory communication?

INTER-THREAD COMMUNICATION

2. Why is `a = a + 1` not atomic?

INTER-THREAD COMMUNICATION

3. **[Prof]** What is a race condition?

INTER-THREAD COMMUNICATION

4. **[Prof]** What happens if two or more threads access a critical region without any thread-safe mechanism?

INTER-THREAD COMMUNICATION

5. What is a critical region?

INTER-THREAD COMMUNICATION

6. What is a mutex?

INTER-THREAD COMMUNICATION

7. How do `lock()` and `unlock()` protect shared data?

INTER-THREAD COMMUNICATION

8. Why is manually calling `lock()` and `unlock()` risky?

INTER-THREAD COMMUNICATION

9. What is RAII locking?

INTER-THREAD COMMUNICATION

10. What is `std::lock_guard`?

INTER-THREAD COMMUNICATION

11. What is `std::unique_lock`, and why is it more flexible?

INTER-THREAD COMMUNICATION

12. What is busy waiting?

INTER-THREAD COMMUNICATION

13. Why is busy waiting usually inefficient?

INTER-THREAD COMMUNICATION

14. What is the producer-consumer problem?

INTER-THREAD COMMUNICATION

15. What is a condition variable?

INTER-THREAD COMMUNICATION

16. Why must condition variables be used with a predicate?

INTER-THREAD COMMUNICATION

17. What is a spurious wakeup?

INTER-THREAD COMMUNICATION

18. How does `wait()` release and reacquire a mutex?

INTER-THREAD COMMUNICATION

19. What is an atomic variable?

INTER-THREAD COMMUNICATION

20. When are atomics appropriate?

INTER-THREAD COMMUNICATION

21. Why are mutexes still needed for complex shared structures like queues?

INTER-THREAD COMMUNICATION

22. How does the final producer-consumer solution combine mutexes and condition variables?